LABORATORIO DI PROGRAMMAZIONE I

Lezione 6

La gestione degli errori

e controllo dell'input

Laura Nenzi

Le eccezioni

In Python gli errori si chiamano “eccezioni”:

```
1
2   my_var = 'Ciao'
3   print(mia_variablie)
4
```

```
$ python esercizio.py
Traceback (most recent call last):
  File "esercizio.py", line 2, in <module>
 print(mia_variablie)
          ^^^^^^^^^^^^^
NameError: name 'mia_variablie' is not defined
```

```
1
2   my_var = 'Ciao'
3   float(my_var)
4
```

```
$ python esercizio.py
Traceback (most recent call last):
  File "esercizio.py", line 2, in <module>
    float(my_var)
ValueError: could not convert string to float: 'Ciao'
```

Cosa sono le eccezioni

- Sono una qualsiasi interruzione del normale flusso del codice
- Segnalano la presenza di un problema che il programma deve gestire esplicitamente.
- Non tutte le eccezioni sono errori, ma tutti gli errori gestiti da Python sono rappresentati come eccezioni.

Cosa sono le eccezioni

Le eccezioni sono oggetti. Come tutto in Python.

Ci sono diversi tipi di eccezioni → *estendono tutti la classe base "Exception"*

Esempi di Built-in Exceptions:

```
Exception
    ArithmeticError
        FloatingPointError
        ZeroDivisionError
    AttributeError
    SyntaxError
    NameError
    TypeError
    ValueError
```

Gerarchia delle Eccezioni

```
BaseException
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError
│   │   └── OverflowError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   └── ...
├── SystemExit
├── KeyboardInterrupt
└── GeneratorExit
```

Quando si verifica una eccezione …

- Python interrompe l'esecuzione del codice dove è avvenuta e stampa il **traceback**

```
1
2   my_var = 'Ciao'
3   float(my_var)
4
```

```
$ python esercizio.py
Traceback (most recent call last):
  File "esercizio.py", line 2, in <module>
    float(my_var)
ValueError: could not convert string to float: 'Ciao'
```

Quando si verifica una eccezione …

- Python interrompe l'esecuzione del codice dove è avvenuta e stampa il **traceback**

```
1
2   my_var = 'Ciao'
3   float(my_var)
4
```

```
$ python esercizio.py
Traceback (most recent call last):
  File "esercizio.py", line 2, in <module>
    float(my_var)
ValueError: could not convert string to float: 'Ciao'
```

File dove è avvenuta l'eccezione, numero di riga e nome funzione

Codice sorgente che corrisponde alle riga

Il tipo ed il messaggio dell'eccezione

Il Traceback

- È un “report” automaticamente generato da Python
- Serve ad identificare dove è avvenuta l’eccezione.
- Quello che fa è proprio di rintracciare all’indietro le chiamate delle funzioni che hanno portato al suo verificarsi, cioè risale lo stack delle chiamate fino a trovare chi può gestirla.

Come si lancia e cattura una eccezione?

- Abbiamo quindi due aspetti essenziali:
  - Come si *lancia una eccezione* per unificare una condizione eccezionale (errore in lettura, chiave mancante)
  - Come si *cattura una eccezione* per decidere come gestirla

Come catturarle

- Possiamo gestire le eccezioni come segue:

Stiamo indicando che il seguente codice può generare una eccezione

```
try:
    codice_che_può_lanciare_eccezione
except ClasseDellEccezione:
    gestione_eccezione
```

Indichiamo che il codice che segue è da eseguire se incontriamo una eccezione di tipo “ClasseDellEccezione”

E loro gestione

- “Risaliamo” lo stack fino a trovare un blocco try…except in grado di gestire l’eccezione
- Possiamo avere più clausole “except” con classi diverse
- Verrà usato il codice nella prima che fa match
- Se facciamo “except A” e B è sottoclasse di A allora abbiamo un match…
- Quindi usiamo le clausole except sempre dalla classe più specifica a quella più generica

Come lanciarle/sollevarle

- Possiamo sollevare una eccezione come segue:

Stiamo indicando che l'oggetto che segue indica l'eccezione generata, tipi diversi di oggetto per eccezioni differenti

```
raise ClasseEccezione()
```

Possiamo definire noi una classe specifica (magari ereditando da "Exception") o usare delle classi predefinite

Alcune note

- La classe *Exception* ha un metodo “*add_note(note)*” che serve per aggiungere informazioni all’eccezione (e.g., un file non può essere aperto e indichiamo il nome del file, etc)
- Possiamo rilanciare l’eccezione all’interno di “except” chiamando **raise** (senza nulla dopo)
- Possiamo dare un nome all’eccezione catturata:
  except ClasseEccezione as nome:

Il costrutto try-except

Permette di gestire le eccezioni (errori).

```
1
2   my_var = 'Ciao'
3
4   try:
5       my_var = float(my_var)
6   except:
7       print('Non posso convertire "my_var" a valore numerico!')
8
```

```
> python lezione5.py
Non posso convertire "my_var" a valore numerico!
```

Il costrutto try-except

Permette di gestire le eccezioni (errori).

```
1
2   my_var = 'Ciao'
3
4   try:
5       my_var = float(my_var)
6   except:
7       print('Non posso convertire "my_var" a valore numerico!')
8       print('Uso il valore di default "0.0" per "my_var"')
9       my_var = 0.0
10
```

```
> python lezione5.py
Non posso convertire "my_var" a valore numerico!
Uso il valore di default "0.0" per "my_var"
```

Il costrutto try-except

...e posso avere l’eccezione dentro l’ except:

```
1
2   my_var = 'Ciao'
3
4   try:
5       my_var = float(my_var)
6   except Exception as e:
7       print('Non posso convertire "my_var" a valore numerico!')
8       print('La variabile "my_var" valeva: "{}"'.format(my_var))
9       print('Ed ho avuto questo errore: "{}"'.format(e))
10
```

```
> python lezione5.py
Non posso convertire "my_var" a valore numerico!
La variabile "my_var" valeva: "Ciao"
Ed ho avuto questo errore: "could not convert string to float: 'Ciao'"
```

Il costrutto try-except

...e posso avere l’eccezione dentro l’ except:

```
1
2   my_var = 'Ciao'
3
4   try:
5       my_var = float(my_var)
6   except Exception as e:
7       print('Non posso convertire "my_var" a valore numerico!')
8       print('La variabile "my_var" valeva: "{}"'.format(my_var))
9       print('Ed ho avuto questo errore: "{}"'.format(e))
10
```

Per usare l'eccezione dentro l’except, devo sempre specificare che eccezione voglio gestire, se le voglio gestire tutte allora uso la classe base Exception

```
> python lezione5.py
Non posso convertire "my_var" a valore numerico!
La variabile "my_var" valeva: "Ciao"
Ed ho avuto questo errore: "could not convert string to float: 'Ciao'"
```

Il costrutto try-except

Posso infatti anche gestire solo eccezioni specifiche:

```
try:

    my_var = float(my_var)

except ValueError:
    print('Non posso convertire "my_var" a valore numerico!')
    print('Ho avuto un errore di VALORE. "my_var" valeva "{}".'.format(my_var))

except TypeError:
    print('Non posso convertire "my_var" a valore numerico!')
    print('Ho avuto un errore di TIPO. "my_var" era di tipo "{}".'.format(type(my_var)))

except Exception as e:
    print('Non posso convertire "my_var" a valore numerico!')
    print('Ho avuto un errore generco: "{}"'.format(e))
```

Il costrutto try-except-else-finally

Permette di gestire le eccezioni (errori).

```
try:
    # Blocco di codice in cui possono verificarsi eccezioni
except [EccezioneTipo] [as e]:
    # Blocco di codice per gestire l'eccezione specificata
else:
    # Blocco di codice che viene eseguito se NON si verifica alcuna eccezione
finally:
    # Blocco di codice che viene sempre eseguito, sia in caso di eccezione che no
```

L'else ed il finally

Sono utili per "fare pulizia"

```
try:
    file_parametro = open(nome_file_parametro)
    parametro_come_stringa = file_parametro.read()
    parametro_come_float = float(parametro_come_stringa)
except IOError:
    print('Non posso leggere il file!')
    print('Userò il vaore di default per il parametro')
except ValueError:
    print('Non posso convertire il parametro a valore numerico!')
    print('Errore di valore, il parametro valeva "{}".'.format(parametro_come_stringa))
    print('Userò il valore di default per il parametro')
except Exception as error:
    print('Errore generico: "{}".'.format(error))
    print('Userò il valore di default per il parametro')
else:
    parametro = parametro_come_float
finally:
    file_parametro.close()
```

Cosa sono gli input

Gli input sono qualsiasi cosa “entri” da qualche parte. Ad esempio:

- gli argomenti delle funzioni, incluse quelle degli oggetti
- i dati caricati da file
- gli input dell’utente (da linea di comando o da richiesta del programma)
- i dati caricati da database (non li vedremo)
- i dati che arrivano dai socket di rete (non li vedremo)
- etc.

Controllo dell'input

Input dell’utente

Abbiamo già visto che in Python esiste una funzione predefinita chiamata `input` che sospende il programma ed attende che l’utente scriva qualcosa

```
>>> testo = input()
Cosa stai aspettando
>>> testo
'Cosa stai aspettando'
```

Prima dell’inserimento dei dati, è buona norma visualizzare un messaggio, chiamato prompt, che informa l’utente di ciò che deve inserire.

```
>>> nome = input('Come...ti chiami?\n')
Come...ti chiami?
Artù, Re dei Bretoni!
>>> nome
'Artù, Re dei Bretoni!'
```

Non fidarsi degli input

- Non ci si deve mai fidare della bontà degli input, perché non si ha il controllo su chi o cosa li genera.

- Se i dati sono in formato sbagliato, danneggiati, fuori range o in generale non adatti per l'uso che se ne deve fare, il programma può:
  - andare in errore
  - oppure, peggio, non mostrare nessun segno di problema ma dare risultati sbagliati.

Cosa vuol dire che un’input è corretto

→ **Di base, *dipende da cosa volete farci***

Ad esempio, se state scrivendo una funzione che legge i dati di un file CSV e volete solo le prime “**n**” righe, questo parametro dovrà:

1. essere un numero
    - oppure una stringa convertibile a numero
    - essere un numero maggiore di zero

Se invece volete specificare un range di righe (**inizio**, **fine**), chiaramente l’inizio dovrà essere antecedente alla fine, ovvero inizio < fine.

Prevedere gli *edge cases*

- Non ci si può basare solo su scenari idealizzati, ma bisogna essere pragmatici e preparati.

- È importante considerare scenari inaspettati, come input vuoti, valori estremamente grandi o piccoli, formati inaspettati.

- È di solito sulle eventuali condizioni rare che in genere sorgono la maggior parte dei problemi.

Prevenire o correggere?

- In genere, i linguaggi tipizzati (come C e Java) preferiscono prevenire tutto
- In Python, che non è un linguaggio tipizzato, è invece in genere preferita la logica del "prima provo, poi chiedo scusa".
- Non si cerca sempre di prevenire, ma piuttosto di correggere gli errori (cioè, gestendo le eccezioni) laddove si verifichino.
- Tuttavia, anche in Python può spesso avere molto senso controllare prima alcune cose. E.g. lavorare con un parametro che deve essere dentro ad un certo range.

Come si controlla che gli input siano corretti

Operatori classici

- >, <, >=, etc.

Controllo di non nullità

- `is` None
- `is not` None
- `== ''` (stringa vuota)

Come si controlla che gli input siano corretti

Valori entro un insieme predefinito

- `sesso in ['M', 'F']`

Tipi, oggetti, classi

- `type(nome_variabile)`
- `isinstance(nome_variabile, Classe)`
- `issubclass(nome_super_classe, nome_sotto_classe)`

Cosa fare se l’input non è corretto

Opzioni possibili:

1) stampare l’errore ed usare un default o aggirarlo (errore “recoverable”)
2) stampare l’errore ed uscire (errore non “recoverable”)

Il punto 2 si fa SOLO se si sta scrivendo il corpo di un programma interattivo.

→ Ovvero, **MAI** nelle **funzioni**, **MAI** negli **oggetti**.

..ma esiste anche

3) generare (sollevare) una nostra eccezione
   → Nelle funzioni e negli oggetti si fa questo!

Come si controlla che gli input siano corretti

Ricordatevi che potete anche “prima provare e poi chiedere scusa”. Esempi:

```
try:
    float(numero)

except:
    # gestite l’errore
```

```
try:
    my_string.split(',')

except:
    # gestite l’errore
```

Ricordiamo come si “solleva” una eccezione

```
raise Exception('Messaggio di errore')
```

Come si “solleva” una eccezione

```
raise Exception('Messaggio di errore')
```

Questa è la stringa che viene poi stampata a schermo quando l'eccezione sale fino all’interprete Python, o che stampate a schermo esplicitamente voi quando le catturate.

Come si “alza” una eccezione

```
raise Exception('Messaggio di errore')
```

..ma MOLTO meglio fare qualcosa del tipo:

```
raise Exception('Ho avuto un errore, ecco il parametro che
                 lo ha generato: "{}"'.format(parametro))
```

Come creare una vostra eccezione

```
class Errore(Exception)
    pass
```

..e poi semplicemente:

```
raise Errore('Mona!')
```

Come creare una vostra eccezione

Esempio di generazione di eccezione:

```
parametro = -5
if parametro < 0:
    raise  ValueError('Il parametro è minore di zero')
```

Esempio di definizione e generazione di una eccezione:

```
class InvalidParameter(Exception):
    pass

parametro = -5
if parametro < 0:
    raise  InvalidParameter('Il parametro è minore di zero')
```

Come si controlla che gli input siano corretti

Esempio: Controllo a priori del tipo di input di una funzione che somma tutti gli elementi di una lista

```
def somma_lista(lista_input):
    if not type(lista_input) == list:
        raise TypeError('Il tipo di lista_input non è lista ' +
            'ma "{}"'.format(type(lista_input)))
    somma = 0
    for elemento in lista_input:
        somma += elemento
    return somma
```

Come si controlla che gli input siano corretti

Esempio: Controllo a priori del tipo di input di una funzione che somma tutti gli elementi di una lista accettando anche le sottoclassi del tipo dati lista.

```
def somma_lista(lista_input):
    if not isinstance(lista_input, list):
        raise TypeError('Il tipo di lista_input non è lista, ' +
                        'ma "{}"'.format(type(lista_input)))
    somma = 0
    for elemento in lista_input:
        somma += elemento
    return somma
```

Sanitizzare gli input

Spesso potete provare a “sanitizzare” gli input, per prevenire potenziali errori.

Esempio 1: supponiamo vi arrivi la variabile **sesso** con il valore “m”.

Se avete fatto il check con:

```
if sesso not in ['M', 'F']:
    # avvisa dell’errore
```

...allora il check non passa. Per evitare questo problema, potete fare:

```
sesso_sempre_maiuscolo = sesso.upper()
if sesso_sempre_maiuscolo not in ['M', 'F']:
    # avvisa dell’errore
```

Sanitizzare gli input

Spesso potete provare a “sanitizzare” gli input, per prevenire potenziali errori.

Esempio 2: supponiamo vi arrivi la variabile **sesso** con il valore “M ”.

Notate lo spazio

Se avete fatto il check con:

```
if sesso not in ['M', 'F']:
    # avvisa dell’errore
```

...allora il check non passa. Per evitare questo problema, potete fare:

```
sesso_pulito = sesso.strip()
if sesso_pulito not in ['M', 'F']:
    # avvisa dell’errore
```

Messaggio da portare a casa

Ragionate sempre in logica “*cosa succede se*”...

Ad esempio:

- non trovo il file che dovrei leggere
- mi passano una stringa e non un numero
- mi chiedono di leggere le righe di un file dalla 100 alla 200 ma il file di righe ne ha solamente 17
- mi passano un valore non ammissibile

Esercizio 1

Modificate l'oggetto **`CSVFile`** della lezione precedente in modo che stampi a schermo un messaggio di errore* se si cerca di aprire un file non esistente.

Potete fare questo controllo:

a) nella funzione `get_data()`, oppure

b) nell' `__init__()` (basta che leggiate la prima riga per vedere se il file esiste)

Esercizio 2

Estendete l'oggetto **`CSVFile`** chiamandolo **NumericalCSVFile** e facendo in modo che converta automaticamente a numero `float` *tutte le colonne* tranne la prima (della data). Chiamate la `get_data` originale con `super().get_data()`, poi converite tutto a float.

A questo punto, aggiungete a mano questi due campi al file "shampoo_sales.csv":

```
01-01-2015,
01-02-2015,ciao
```

e gestite gli errori che verranno generati in modo che le linee vengano saltate senza bloccare il programma ma che venga stampato a schermo l'errore

Esercizio 3

Create un oggetto **CSVFile** che rappresenti un file CSV, e che:

1) venga inizializzato sul nome del file csv, e
2) abbia un attributo "`name`" che ne contenga il nome
3) abbia un metodo "`get_data()`" che torni i dati dal file CSV come lista di liste, ad es: `[ ['01-01-2012', '266.0'], ['01-02-2012', '145.9'], ... ]`

Esercizio 4

Modificate l’oggetto **CSVFile** della lezione precedente in modo che alzi un'eccezione se il nome del file non è una stringa (nell’ `__init__()`).

Poi, modificate la funzione **get_data**() di **`CSVFile`** in modo da leggere solo un’ intervallo di righe del file*, aggiungendo gli argomenti `start` ed `end` *opzionali*, controllandone la correttezza e sanitizzandoli se serve.

```
get_data(self, start=None, end=None)
```

*inclusa l’intestazione, estremi inclusi, e partendo da “1”.

Esercizio 5

Nota: se avete eseguito correttamente l'esercizio 2, anche `NumericalCSVFile` ha quasi automaticamente ereditato queste migliorie.

"Quasi" perchè mentre il controllo del nome è stato effettivamente ereditato, per il range di righe bisogna ricorrere ad una "formula magica" che "inoltra" gli argomenti della funzione figlia al padre, a prescindere da quali essi siano:

```
class NumericalCSVFile(CSVFile):
            def get_data(self, *args, **kwargs):
            csv_data = super().get_data(*args, **kwargs)
                    ...
```

Esercizio 6

Scrivete un programma che riceva una data di nascita come input e visualizzi l’età dell’utente e il numero di giorni, ore, minuti e secondi che mancano al prossimo compleanno.

Esercizio 7

Scrivete un programma che chieda all'utente di inserire un numero intero. Se l'utente inserisce un valore valido, il programma deve stampare il quadrato del numero. Se l'utente inserisce un valore non valido, il programma deve visualizzare un messaggio di errore e richiedere nuovamente l'input.

Esercizio 8

Create un programma che mostri un menù all'utente con tre opzioni:

1. Calcolare la somma di due numeri.
2. Calcolare la differenza tra due numeri.
3. Uscire.

Il programma deve:

1. Chiedere all'utente di scegliere un'opzione (1, 2 o 3).
2. Eseguire l'operazione corrispondente se l'input è valido.
3. Gestire input non validi mostrando un messaggio di errore.
4. Continuare a mostrare il menù fino a quando l'utente sceglie di uscire (opzione 3)